



FUNDAMENTOS Y TEORÍA DE LA COMPUTACIÓN

COMPILADORES

Traductores:

Hace unos años atrás, se utilizaban los lenguajes de primera generación que operan a nivel de código binario de la máquina, o sea una secuencia de ceros y unos con los que se instruye al ordenador para que realice acciones.

El código de máquina fue reemplazado por los lenguajes de segunda generación, o lenguajes ensambladores. Estos lenguajes permiten usar abreviaturas nemónicos como nombres simbólicos.

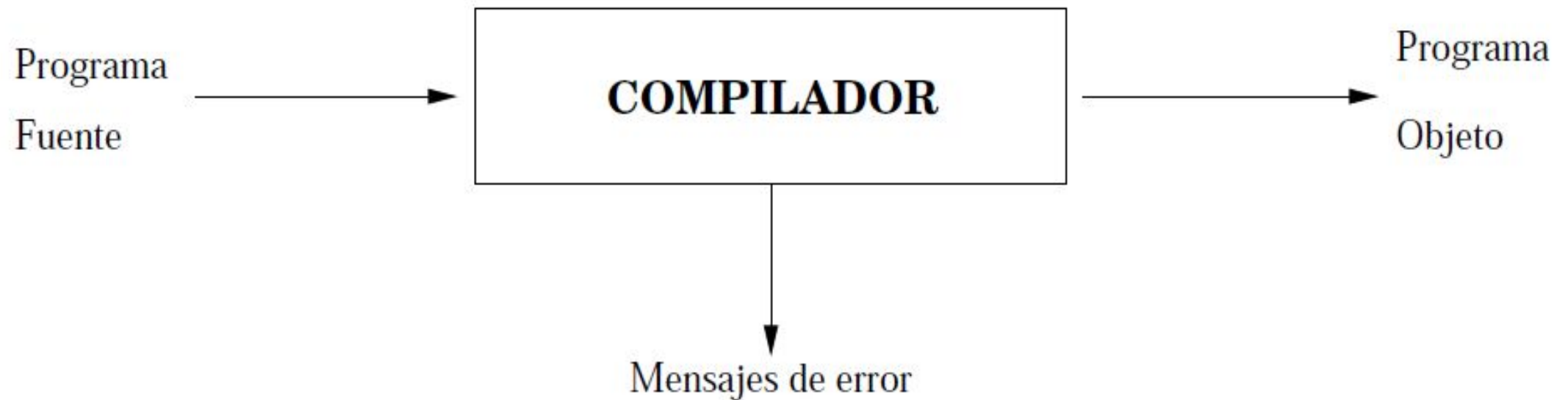
Luego llegaron los lenguajes de tercera generación o lenguajes de alto nivel. Con ellos se pueden usar estructuras de control basadas en objetos de datos lógicos: variables de un tipo específico. Ofrecen un nivel de abstracción que permite la especificación de los datos, funciones o procesos independiente de la máquina.

COMPILADORES

Traductores (cont.):

Para ejecutar programas en lenguajes de alto nivel, éstos deben ser traducidos a código máquina. A este proceso se le denomina compilación, y la herramienta correspondiente se llama compilador.

Un compilador es un programa que lee otro, escrito en lenguaje fuente, y lo traduce a lenguaje objeto:



COMPILADORES

Traductores y compiladores:

Un traductor es un programa que acepta cualquier texto expresado en un lenguaje (el lenguaje fuente del traductor) y genera un texto semánticamente equivalente expresado en otro lenguaje (su lenguaje destino).

Un ensamblador traduce un lenguaje ensamblador en su correspondiente código máquina. Generalmente, un ensamblador genera una instrucción de la máquina por cada instrucción fuente.

Un compilador traduce desde un lenguaje de alto nivel a otro lenguaje de bajo nivel. Generalmente, un compilador genera varias instrucciones de la máquina por cada comando fuente.

COMPILADORES

Intérpretes:

Un intérprete es un programa que acepta otro programa (el programa fuente) escrito en un determinado lenguaje (el lenguaje fuente), y ejecuta el programa inmediatamente.

Un intérprete trabaja cargando, analizando y ejecutando una a una las instrucciones del programa fuente.

El programa fuente comienza a ejecutarse y produce resultados desde el momento en que la primera instrucción ha sido analizada.

El intérprete no traduce el programa fuente en un código objeto.

Uso de intérpretes:

La interpretación es un buen método cuando se dan las siguientes circunstancias:

- El programador está trabajando en forma interactiva, y quiere ver el resultado de cada instrucción antes de entrar la siguiente instrucción.
- El programa se va a utilizar solo una vez, y por tanto la velocidad de ejecución no es importante.
- Se espera que cada instrucción se ejecute una sola vez.
- Las instrucciones tiene un formato simple, y por tanto pueden ser analizadas de forma fácil y eficiente.

Uso de intérpretes (cont.):

La interpretación de un programa fuente puede ser 100 veces más lenta que la ejecución del programa equivalente escrito en código máquina.

La interpretación no es interesante cuando:

- El programa se va a ejecutar en modo de producción, y por tanto la velocidad es importante.
- Se espera que las instrucciones se ejecuten frecuentemente.
- Las instrucciones tienen formatos complicados, y por tanto su análisis es costoso en tiempo.

Intérpretes conocidos:

- 1) Intérprete Caml: Caml es un lenguaje funcional. El intérprete lee cada línea hasta el símbolo ";" y la ejecuta produciendo una salida.
- 2) Intérprete Lisp: Lisp es un lenguaje en el que existe una estructura de datos tanto para el código como para los datos.
- 3) Intérprete de comandos de Unix (shell): Una instrucción para el sistema operativo Unix se introduce dando el comando de forma textual.
- 4) Intérprete SQL: El usuario extrae información de la base de datos introduciendo una consulta SQL.

COMPILADORES

Compiladores interpretados:

Un compilador puede tardar mucho en traducir un programa fuente, pero una vez hecho esto, el programa puede correr a la velocidad de la máquina.

Un intérprete permite que el programa comience a ejecutarse inmediatamente, pero corre muy lento.

Un compilador interpretado es una combinación de compilador e intérprete, reuniendo algunas de las ventajas de cada uno de ellos. La idea principal es traducir el programa fuente en un lenguaje intermedio.

El código de la Máquina Virtual de Java (el JVM-code) es un lenguaje intermedio orientado a Java.

COMPILADORES

Contexto de un compilador:

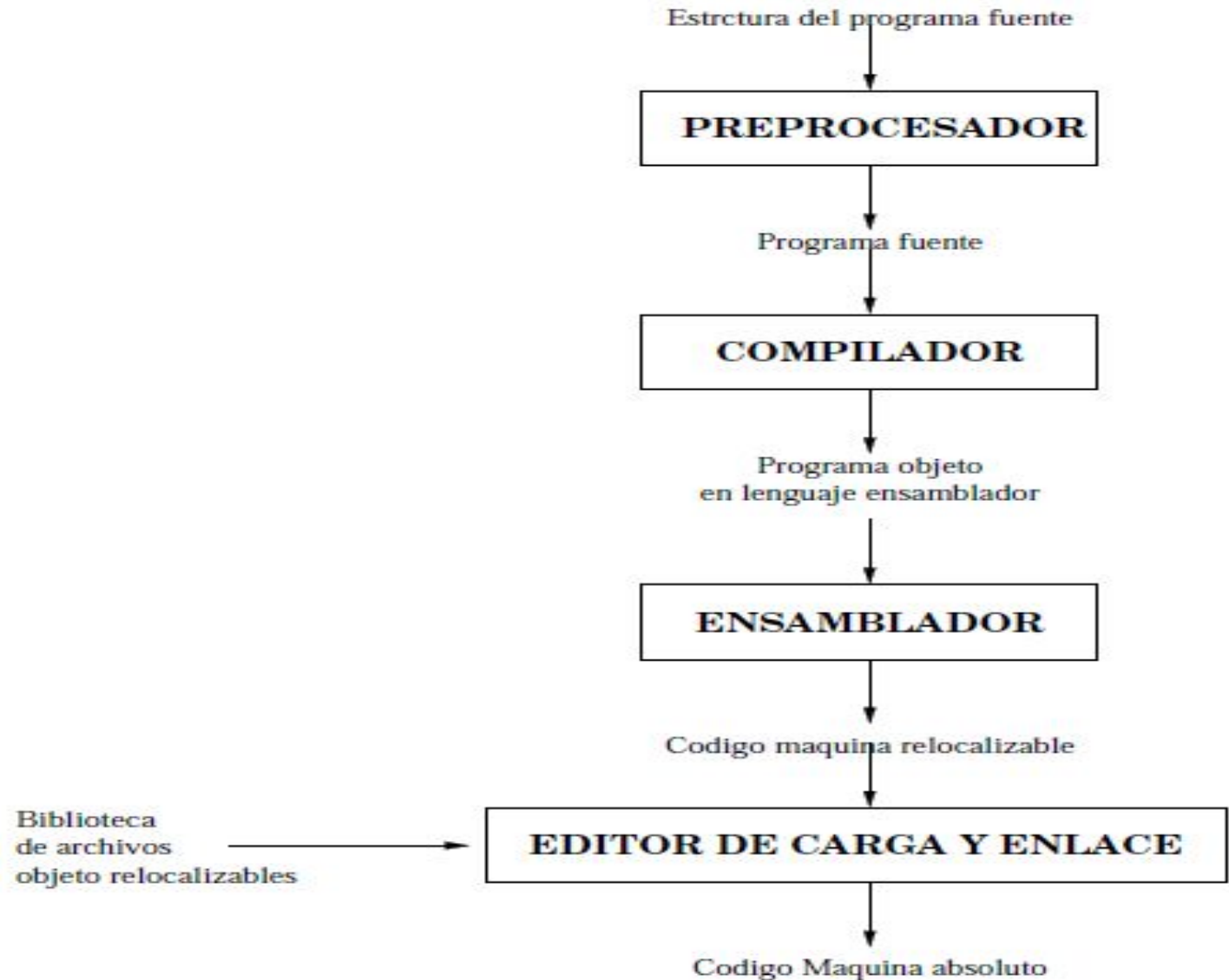
En el proceso de construcción de un programa escrito en código máquina a partir del programa fuente, suelen intervenir, aparte del compilador, otros programas:

- Preprocesador: Es un traductor cuyo lenguaje fuente es una forma extendida de algún lenguaje de alto nivel, y cuyo lenguaje objeto es la forma estándar del mismo lenguaje. Por ejemplo C.
- Ensamblador: Traduce el programa en lenguaje ensamblador, creado por el compilador, a código máquina.
- Cargador y linkador: Un cargador es un traductor cuyo lenguaje objeto es el código de la máquina real, que consiste usualmente en programas de lenguaje máquina en forma reubicable, junto con tablas de datos que especifican los puntos en donde el código reubicable debe modificarse para convertirse en verdaderamente ejecutable. Un linkador toma como entrada programas en forma reubicable que se han compilado separadamente, incluyendo subprogramas almacenados en librerías, los une en una sola unidad de código máquina lista para ejecutarse.

COMPILADORES

Contexto de un compilador (cont.):

En general, un editor de carga y enlace une el código máquina a rutinas de librería para producir el código que realmente se ejecuta en la máquina:



COMPILADORES

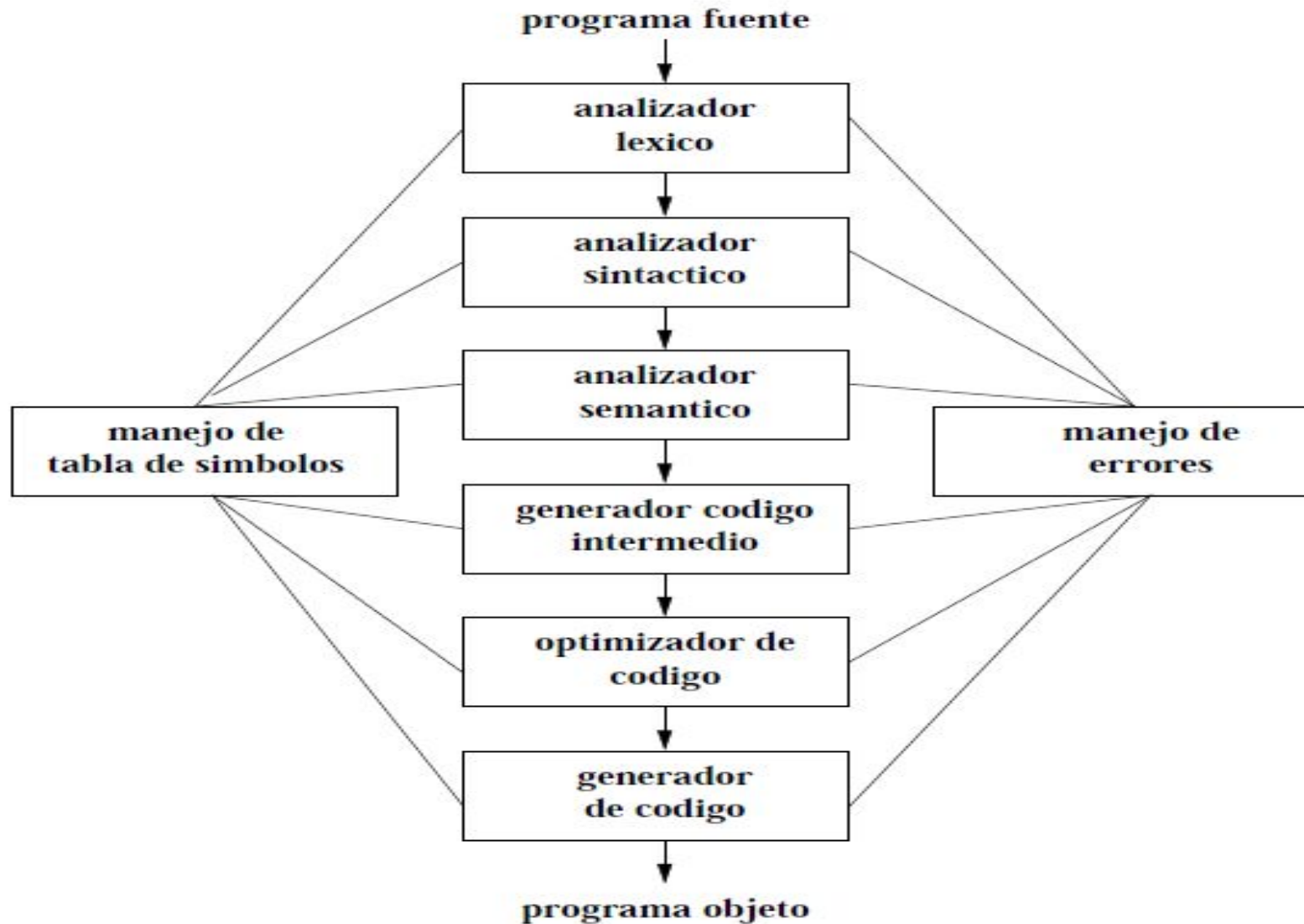
Fases y estructura de un compilador:

La compilación consiste en dos tareas bien diferenciadas:

- 1) **Análisis**: Se determina la estructura y el significado de un código fuente. Esta parte del proceso de compilación divide al programa fuente en sus elementos componentes y crea una representación intermedia de él, llamada árbol sintáctico.
- 2) **Síntesis**: Se traduce el código fuente a un código de máquina equivalente, a partir de esa representación intermedia. Aquí, es necesario usar técnicas mas especializadas que durante el análisis.

Conceptualmente, un compilador opera en estas dos etapas, que a su vez pueden dividirse en varias fases.

COMPILADORES



COMPILADORES

Fases y estructura de un compilador (cont.):

Las tres primeras fases conforman la mayor parte de la tarea de análisis en un compilador, mientras que las tres últimas pueden considerarse como constituyentes de la parte de síntesis del mismo.

Durante el análisis léxico, la cadena de caracteres que constituye el programa fuente, se lee de izquierda a derecha, y se agrupa en componentes léxicos.

En el análisis sintáctico, los componentes léxicos se agrupan jerárquicamente en colecciones anidadas con un significado común.

En la fase de análisis semántico se realizan ciertas revisiones para asegurar que los componentes de un programa se ajustan de un modo significativo.

Las tres últimas fases suelen variar de un compilador a otro. Existen, por ejemplo, compiladores que no generan código intermedio, o no lo optimizan.

De manera informal, también se consideran fases al administrador de la tabla de símbolos y al manejador de errores, que están en interacción con todas las demás.

COMPILADORES

Administración de la tabla de símbolos:

Un compilador debe registrar los identificadores utilizados en el programa fuente y reunir información sobre los distintos atributos de cada identificador. Estos atributos pueden proporcionar información sobre la memoria asignada a un identificador, su tipo, su ámbito, etc.

Una tabla de símbolos es una estructura de datos que contiene un registro por cada identificador, con campos para los atributos del identificador.

Cuando el analizador léxico detecta un identificador en el programa fuente, este identificador se introduce en la tabla de símbolos. Normalmente los atributos de un identificador no se pueden determinar durante el análisis léxico. Las fases restantes introducen información sobre los identificadores en la tabla de símbolos, y después la utilizan de varias formas.

COMPILADORES

Detección e información de errores:

Cada fase dentro de proceso de compilación, puede encontrar errores. Cada fase debe tratar de alguna forma ese error, para poder continuar la compilación, permitiendo la detección de nuevos errores en el programa fuente.

La fase de análisis léxico puede detectar errores donde los caracteres restantes de la entrada no forman ningún componente léxico del lenguaje.

Los errores donde la cadena de componentes léxicos viola las reglas de la estructura del lenguaje (sintaxis) son determinados por la fase de análisis sintáctico.

Durante la fase de análisis semántico, el compilador intenta detectar construcciones que tengan la estructura sintáctica correcta, pero que no tengan significado para la operación implicada (sumar dos identificadores: una matriz y un procedimiento).

Análisis léxico (o lineal):

La tarea básica que realiza el AL es transformar un flujo de caracteres de entrada en una serie de componentes léxicos o tokens.

La secuencia de caracteres que forma el token se denomina lexema.

A un mismo token le pueden corresponder varios lexemas. Por ejemplo, se pueden reconocer como tokens de tipo ID a todos los identificadores. Aunque para analizar sintácticamente una expresión, solo nos haría falta el código de token, el lexema debe ser recordado, para usarlo en fases posteriores dentro del proceso de compilación.

Análisis léxico (cont.):

El AL es el único componente del compilador que tendrá acceso al código fuente. Por lo que debe almacenar los lexemas para que puedan ser usados posteriormente.

Esto se hace en la tabla de símbolos. Además, debe enviar al analizador sintáctico, aparte del código de token reconocido, la información del lugar donde se encuentra almacenado ese lexema (por ejemplo, mediante un apuntador a la posición que ocupa dentro de la tabla de símbolos).

Posteriormente, en otras fases del compilador, se irá completando la información sobre cada ítem de la tabla de símbolos.

Análisis léxico (cont. 2):

Por ejemplo, ante la sentencia de entrada

costo = precio * 0,98;

El AL podría devolver una secuencia de parejas, como la siguiente:

[ID,1] [=,] [ID,2] [*,] [CONS,3] [;,]

Donde ID, =, *, CONS y ; corresponderían a códigos de tokens y los números a la derecha de cada pareja serían índices de la tabla de símbolos.

Análisis léxico (cont. 3):

Si durante la fase de análisis léxico, el AL se encuentra con uno o más lexemas que no corresponden a ningún token válido, debe dar un mensaje de error léxico e intentar recuperarse.

Finalmente, puesto que el AL es el único componente del compilador que tiene contacto con el código fuente, debe encargarse de eliminar los símbolos no significativos del programa, como espacios en blanco, tabuladores, comentarios, etc.

Para identificar los tokens del lenguaje, los algoritmos se diseñan en base a los autómatas finitos que reconocen esos tokens.

Análisis sintáctico (o jerárquico):

Su función es revisar si los tokens del código fuente que le proporciona el analizador léxico aparecen en el orden correcto (impuesto por la gramática), y los combina para formar unidades gramaticales, dándonos como salida el árbol de derivación o árbol sintáctico correspondiente a ese código fuente.

Si el programa no tiene una estructura sintáctica correcta, el analizador sintáctico no podrá encontrar el árbol de derivación correspondiente y deberá dar mensaje de error sintáctico.

La mayor parte de los lenguajes de programación pertenecen realmente al grupo de lenguajes dependientes del contexto, y en el diseño del AS se usan autómatas de pila.

Análisis semántico:

La fase de análisis semántico revisa el programa fuente para tratar de encontrar errores semánticos, y reúne la información sobre los tipos para la fase posterior de generación de código.

Utiliza la estructura jerárquica que se construye en la fase de análisis sintáctico, para identificar operadores y operandos de expresiones y proposiciones. Además, accede, completa y actualiza con frecuencia la tabla de símbolos.

Una tarea importante es la verificación de tipos. Aquí, el compilador comprueba si cada operador tiene operandos permitidos por la especificación del lenguaje fuente. A veces esta especificación puede permitir ciertas conversiones de tipos en los operandos.

Análisis semántico (cont.):

Algunas de las comprobaciones que puede realizar, son:

- Chequeo y conversión de tipos.
- Comprobación de que el tipo y número de parámetros en la declaración de funciones coincide con los de las llamadas a esa función.
- Comprobación del rango para índices de arreglos.
- Comprobación de la declaración de variables.
- Comprobación de las reglas de alcance de variables.

Generación de código:

El árbol de derivación obtenido como resultado del análisis sintáctico, junto con la información contenida en la tabla de símbolos, se usa para la construcción del código objeto.

Existen varios métodos para conseguir esto. Uno es el que se conoce como traducción dirigida por la sintaxis. Consiste en asociar a cada nodo del árbol de derivación una cadena de código objeto. El código correspondiente a un nodo se construye a partir del código de sus descendientes y del código que representa acciones propias de ese nodo. Este método es ascendente, pues parte de las hojas del árbol de derivación y va generando código hacia arriba, hasta que llegamos a la raíz del árbol. La raíz representa el símbolo inicial de la gramática y su código asociado será el programa objeto deseado.

Generación de código (cont.):

A veces, el proceso de generación de código se puede dividir en las siguientes fases:

- **Generación de código intermedio**: se genera una representación intermedia explícita del programa fuente. Esta representación intermedia se puede considerar como un programa para una máquina abstracta.
- **Optimización del código**: trata de mejorar el código intermedio, de modo que se obtenga un código máquina más eficiente en tiempo de ejecución.
- **Generación de código objeto**: por lo general consiste en código de máquina reubicable o código ensamblador.

COMPILADORES

Ejemplo:

Tabla de Simbolos

posicion	...
inicial	...
velocidad	...

